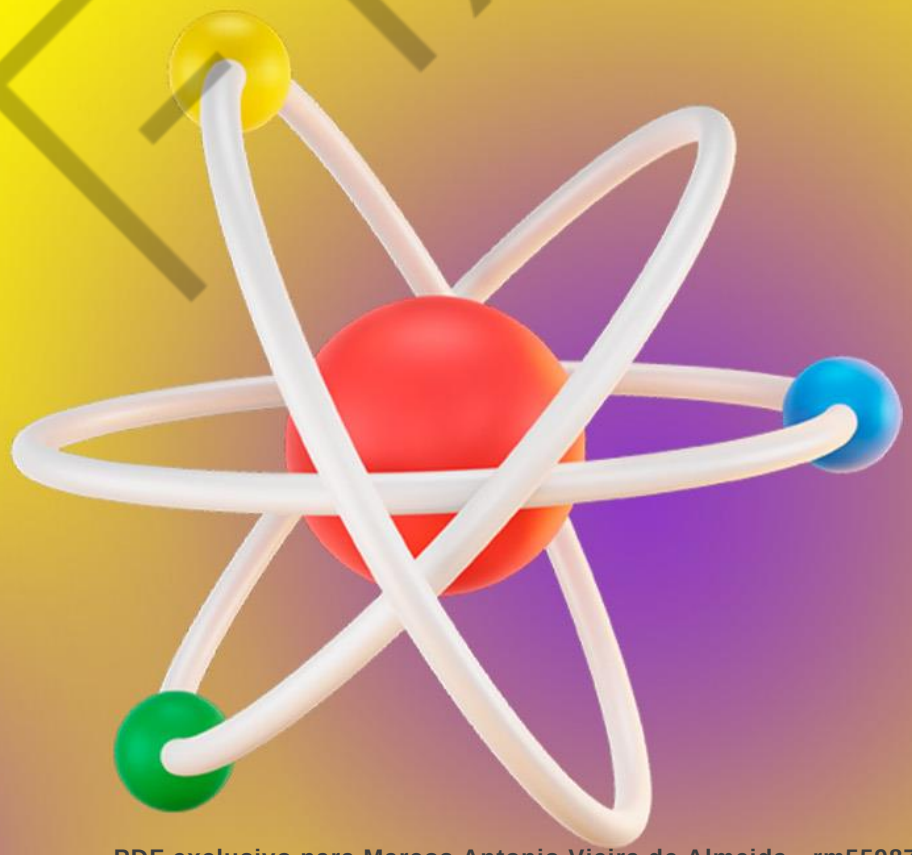
BUILDING STATISTICAL MODELS

REDES NEURAIS RECORRENTES



13

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Imagem de uma bola em movimento em um instante qualquer 5
Figura 2 – Imagens sequenciais de uma bola em movimento indicando sua trajetória 5
Figura 3 – Comparação da arquitetura de uma RNA com uma RNN 8
Figura 4 – Representação de uma RNN desdobrada (unfolded RNN).....9
Figura 5 – Representação do problema de memória de curto prazo 11
Figura 6 – Representação de um neurônio de uma LSTM.....14
Figura 7 – Representação de um neurônio de uma GRU 15
Figura 8 – Representação da estrutura geral de uma rede neural bidirecional.....16
Figura 9 – Representação da estrutura geral de uma rede neural bidirecional.....17
Figura 10 – Saída do Código-fonte 2 24
Figura 11 – Saída do Código-fonte 3 25
Figura 12 – Saída do Código-fonte 4 26
Figura 13 – Saída do Código-fonte 5 26
Figura 14 – Saída do Código-fonte 7 28
Figura 15 – Saída do Código-fonte 8 29
Figura 16 – Saída do Código-fonte 10 31
Figura 17 – Saída do Código-fonte 11 32
Figura 18 – Saída do Código-fonte 13 36

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports do código de LSTM 23
Código-fonte 2 – Carregando o dataset 23
Código-fonte 3 – Selecionando apenas a coluna “Open” 24
Código-fonte 4 – Plotando a série histórica dos dados 25
Código-fonte 5 – Padronizando a escala dos dados entre 0 e 1 26
Código-fonte 6 – Lógica para criar o dataset da série temporal 27
Código-fonte 7 – Reescalando o formato da variável de treino 27
Código-fonte 8 – Definindo, compilando e treinando a LSTM 28
Código-fonte 9 – Lendo e preparando os dados de teste 30
Código-fonte 10 – Predizendo ações a partir da série temporal.....31
Código-fonte 11 – Predizendo ações a partir da série temporal.....31
Código-fonte 12 – Avaliando o RMSE do modelo anterior 33
Código-fonte 13 – Proposta de solução do Hands On 36

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

REDES NEURAIS RECORRENTES.....5
1 INTRODUÇÃO 5
1.1 Importância das RNNs 5
1.2 Aplicações.....6
2 ARQUITETURA DAS REDES NEURAIS RECORRENTES.....8
2.1 Estrutura básica 8
2.2 Forças e Fraquezas 9
2.3 O problema do desaparecimento do gradiente.....10
2.4 Arquiteturas derivadas.....13
3 ARQUITETURA LSTM 17
3.1 Introdução 17
3.2 Componentes das LSTMs.....18
3.3 Fluxo de informações em uma célula LSTM 19
3.4 Melhoras práticas.....20
4 MÃO NA MASSA! IMPLEMENTANDO UMA LSTM 22
5 HANDS ON: PRATICANDO O USO DE LSTM 33
5.1 Seu desafio 33
5.2 Proposta de solução.....33
REFERÊNCIAS.....37
GLOSSÁRIO 38

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REDES NEURAIS RECORRENTES

1 INTRODUÇÃO

1.1 Importância das RNNs

As redes neurais recorrentes (RNNs) são uma classe de algoritmos de aprendizado de máquina especialmente projetados para lidar com dados sequenciais ou dependentes no tempo. Para entender a importância das RNNs, considere uma cena com uma bola em movimento. Se você visse apenas uma foto estática da bola, como na Figura 1, seria difícil prever sua trajetória ou para onde ela está indo, pois existem infinitas possibilidades (destacados algumas delas na Figura 1).

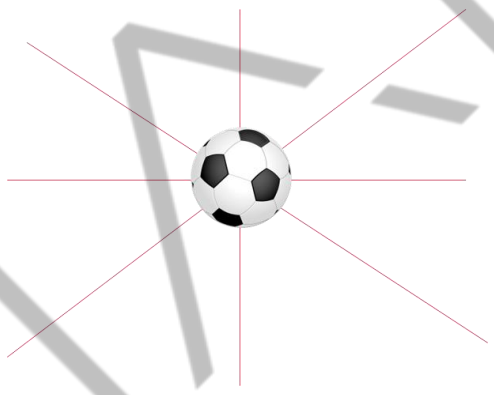


Figura 1 – Imagem de uma bola em movimento em um instante qualquer
Fonte: Produzido pelo autor (2024)

No entanto, se tivéssemos uma sequência de fotos tiradas em momentos diferentes, poderíamos observar a direção da bola ao longo do tempo, permitindo-nos fazer uma previsão mais precisa sobre seu movimento futuro, como na Figura 2.

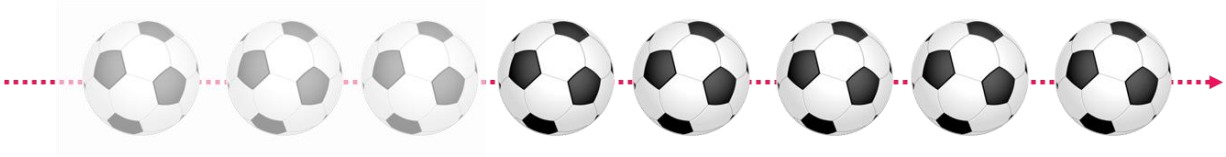


Figura 2 – Imagens sequenciais de uma bola em movimento indicando sua trajetória
Fonte: Produzido pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Da mesma forma, as RNNs são capazes de lidar com dados sequenciais, capturando a dependência temporal entre os elementos da sequência.

Ao contrário das redes neurais convolucionais (CNNs), que são mais adequadas para dados com estrutura espacial, como imagens, **as RNNs são projetadas para lidar com dados sequenciais**, como texto, áudio e muito mais. Essa capacidade de modelar dependências temporais as torna extremamente poderosas em uma variedade de aplicações, incluindo novas formas de interação humano-máquina, algumas análises típicas da Saúde e do setor Financeiro, bem como predição de clima.

A principal característica distintiva das RNNs é sua capacidade de manter estados anteriores e usar essas informações para processar dados subsequentes na sequência. Isso permite que as RNNs capturem dependências de longo alcance em dados sequenciais, o que é crucial em muitas tarefas de aprendizado de máquina.

No entanto, as RNNs também enfrentam desafios, como a própria forma de organizar e estruturar um banco de dados dependente do tempo. Além disso, a necessidade de memorizar sequências longas implica em maior consumo de memória e dados. Para superar esses desafios, foram desenvolvidas variantes das RNNs, como as redes neurais recorrentes de memória de curto e longo prazo (LSTMs), que veremos adiante.

1.2 Aplicações

As RNNs têm se destacado em uma variedade de aplicações de negócios, oferecendo soluções inovadoras e eficazes para uma série de desafios. Uma das áreas mais proeminentes é o **reconhecimento de fala**, onde as RNNs são empregadas para transcrever áudio em texto com alta precisão. Empresas de tecnologia como Google, Amazon e Apple têm investido pesadamente em sistemas de reconhecimento de fala baseados em RNNs, melhorando significativamente a usabilidade de seus assistentes virtuais e dispositivos inteligentes.

Além disso, as RNNs têm sido amplamente utilizadas na **tradução automática**, facilitando a comunicação entre pessoas que falam idiomas diferentes. Plataformas

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

como o Google Translate empregam modelos de tradução baseados em RNNs para oferecer traduções rápidas e precisas em uma ampla variedade de idiomas. Essa aplicação não apenas torna a comunicação global mais acessível, mas também abre novas oportunidades de mercado para empresas que buscam expandir sua presença internacional.

Outro campo de aplicação importante é a **previsão de séries temporais**, onde as RNNs têm se destacado no fornecimento de insights valiosos para empresas de diversos setores. Por exemplo, em finanças, as RNNs são usadas para **prever preços de ações**, permitindo que investidores tomem decisões mais informadas e estratégicas. Da mesma forma, em empresas de varejo, as RNNs são empregadas para **prever demanda de produtos**, otimizando o estoque e reduzindo custos operacionais.

Além dessas áreas, as RNNs encontram aplicação em uma variedade de outros contextos de negócios. Por exemplo, em marketing digital, as RNNs são usadas para **personalizar recomendações de produtos com base no histórico de compras** dos clientes, aumentando a taxa de conversão e o engajamento do usuário. Em medicina, as RNNs são aplicadas para **análise de séries temporais de sinais vitais**, auxiliando médicos no diagnóstico precoce de doenças e na tomada de decisões clínicas mais precisas.

Em resumo, as RNNs oferecem um arsenal versátil e poderoso para resolver uma variedade de problemas de negócios, desde melhorar a experiência do usuário até otimizar processos operacionais e informar decisões estratégicas. Com sua capacidade de lidar com dados sequenciais e capturar dependências temporais complexas, as RNNs estão transformando a forma como as empresas abordam os desafios do mundo real, impulsionando a inovação e impulsionando o sucesso comercial.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 ARQUITETURA DAS REDES NEURAIS RECORRENTES

2.1 Estrutura básica

A estrutura básica de uma RNN consiste em unidades recorrentes conectadas em série, formando loops que permitem que as informações sejam propagadas ao longo da sequência temporal. Visualmente, uma RNN pode ser representada como uma série de blocos conectados, onde cada bloco representa uma unidade recorrente e as conexões entre os blocos indicam a passagem das informações ao longo do tempo.

A Figura 3 compara a representação de uma arquitetura de rede neural artificial feedforward (a) com de uma rede recorrente (b):

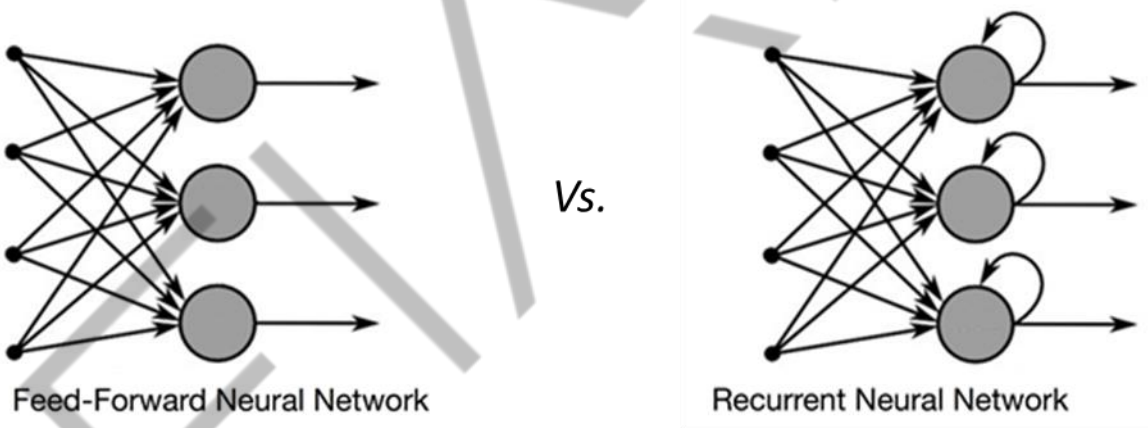


Figura 3 – Comparação da arquitetura de uma RNA com uma RNN
Fonte: Produzido pelo autor (2024)

A Figura 3 mostra que, ao contrário de uma rede neural comum, onde cada entrada é tratada de forma independente, uma RNN processa sequências de dados, mantendo uma memória interna que captura as dependências temporais. Esse processo permite que a rede faça previsões ou classificações com base em informações contextuais, levando em consideração o histórico da sequência.

Uma RNN tradicional opera em um único passo de tempo de cada vez, o que pode levar a problemas de dissipação ou explosão do gradiente durante o

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

treinamento. Para mitigar esses problemas, as RNNs desdobradas (*unfolded RNNs*) foram introduzidas (Figura 4).

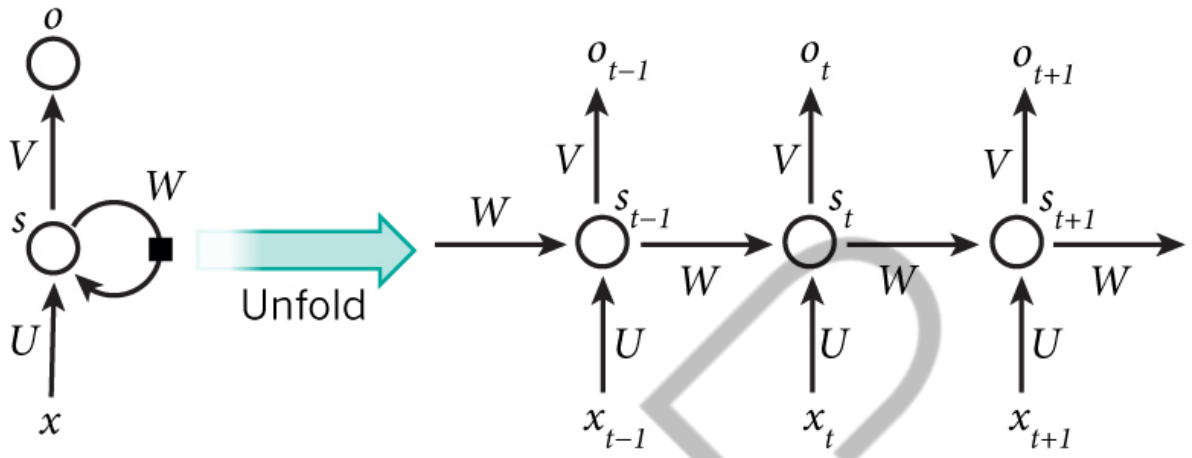


Figura 4 – Representação de uma RNN desdobrada (unfolded RNN)
Fonte: Imagem reproduzida de: https://www.researchgate.net/figure/Figure-of-unfolded-RNN_fig1_328281924

Nessa abordagem, a rede é expandida em múltiplos passos de tempo, criando uma representação gráfica que se estende ao longo da sequência temporal. Isso permite que as informações fluam livremente ao longo da rede, facilitando o treinamento e permitindo que a rede capture relações de longo prazo entre os dados. Durante o treinamento, as RNNs são atualizadas usando o algoritmo de retropropagação no tempo (*Backpropagation Through Time – BPTT*), que é uma extensão do algoritmo de retropropagação usado em redes neurais comuns. Esse processo ajusta os pesos da rede de acordo com o erro calculado em cada passo de tempo, permitindo que a rede aprenda a modelar dependências temporais nos dados de entrada.

2.2 Forças e Fraquezas

As RNNs apresentam diversas **forças** que as tornam poderosas em lidar com dados sequenciais. Uma delas é a capacidade de processar entradas de qualquer tamanho, permitindo flexibilidade no tamanho das sequências de dados que podem ser alimentadas na rede. Além disso, o tamanho do modelo não aumenta com o

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

tamanho da entrada, o que significa que a mesma arquitetura pode ser aplicada a diferentes tamanhos de sequência, economizando recursos computacionais.

A capacidade das RNNs de considerar informações históricas é outra vantagem significativa. Isso significa que a computação em um determinado passo de tempo leva em conta o contexto das etapas anteriores, permitindo que a rede capture dependências temporais nos dados. Além disso, o compartilhamento de pesos ao longo do tempo, presente em arquiteturas derivadas das RNNs, ajuda a reduzir o número de parâmetros do modelo, tornando-o mais eficiente e eficaz em aprender representações temporais.

No entanto, as RNNs também apresentam algumas **fraquezas** importantes. Uma delas é a computação lenta, especialmente em sequências longas, devido à natureza serial do processamento. Além disso, as RNNs podem ter dificuldade em acessar informações de muito tempo atrás devido à limitada capacidade de memória, muitas vezes referida como "memória curta".

Por fim, as RNNs não podem considerar informações futuras para o estado atual, e a incorporação de dados externos pode ser complexa, o que limita sua capacidade de modelar relações complexas em dados sequenciais.

2.3 O problema do desaparecimento do gradiente

Dentre os diversos desafios de trabalhar com RNNs, o principal é o **desaparecimento do gradiente**. Para entender esse fenômeno, primeiramente devemos compreender o conceito de “**memória sequencial**”, que é a base de como as RNNs aprendem. Toda RNN está baseada no princípio da memória sequencial.

A **memória sequencial** refere-se à capacidade de um sistema de armazenar e acessar informações de eventos anteriores em uma sequência de dados. Nós também temos essa memória! Por exemplo, faça o seguinte exercício:

1. Diga o alfabeto na ordem comum.
2. Imagino que o passo [1] foi fácil! Agora, diga o alfabeto na ordem inversa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3. Acredito que o passo [2] foi bem mais complicado! Então, diga o alfabeto a partir da letra “F”.
4. Imagino que você precisou pensar um pouco para chegar à letra “F”; depois, as próximas letras vieram naturalmente na sua mente.

A memória sequencial nos ajuda a recuperar informações em ordens esperadas, o que confere velocidade nas respostas. Outro exemplo simples é a previsão do próximo caractere em uma sequência de palavras. Por exemplo, ao tentar prever a próxima letra da palavra "*hello*", saber que a letra "h" é seguida por "e", que por sua vez é seguida por "l", é crucial para prever com precisão que a próxima letra é "l".

A capacidade conferida pela memória sequencial é relevante para as RNNs porque viabiliza a captura de dependências nos dados, que podem ser exploradas pela rede para predições futuras.

O problema ocorre quando, para compreender um contexto, tentamos (ou precisamos!) acumular muita informação ou necessitamos da informação mais antiga. A Figura 5 ilustra essa questão.

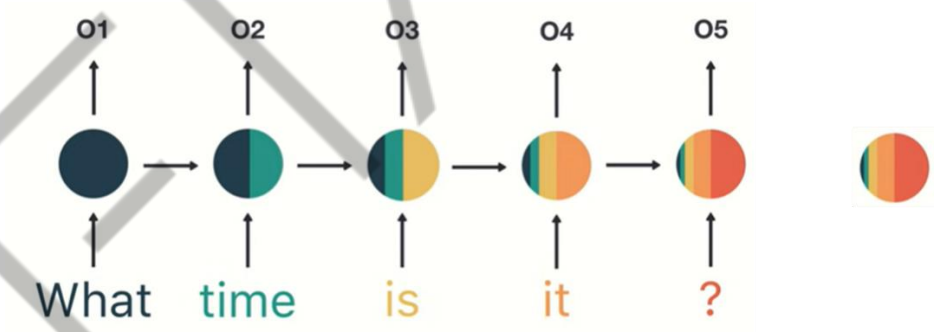


Figura 5 – Representação do problema de memória de curto prazo
Fonte: Produzido pelo autor (2024)

A Figura 5 mostra, de forma simplificada, o acúmulo de informação em um neurônio recorrente para aprender a decifrar o conteúdo da frase “*What time is it?*” (“Que horas são?”, em inglês). Por questões didáticas, vamos manter o “?” na frase. Em um pipeline tradicional de processamento de linguagem natural, qualquer pontuação seria removida. Na primeira interação, o neurônio “aprende” apenas o primeiro item “what”, cuja informação é parcialmente retida e utilizada na próxima

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

interação, aprendendo a combinação “what time”. Essa combinação, por sua vez, influencia o aprendizado do próximo item “is”, e assim sucessivamente.

A cada iteração, notamos que o aprendizado acumulado representa aproximadamente 50% da informação retida e que os outros 50% vêm da informação atual, independentemente de qual seja.

O destaque à direita da Figura 5 ilustra “o aprendizado acumulado” do neurônio em questão ao final do processamento. Note que ~75% do que é “mais importante” para ele é o dado mais recente “is it?”, sendo que o sentido da frase está justamente nos primeiros itens “What time”, que representam ~15% do conteúdo retido.

A memória sequencial leva ao problema da memória de curto prazo, o que gera o **problema do desaparecimento do gradiente**. Esse é uma dificuldade encontrada durante o treinamento de redes neurais profundas, onde os gradientes associados aos pesos das camadas anteriores da rede se tornam cada vez menores à medida que são propagados para trás durante o processo de retropropagação. Esse problema surge devido à multiplicação contínua de gradientes menores que 1 ao longo das camadas da rede, o que faz com que os gradientes diminuam exponencialmente à medida que retrocedem nas camadas.

Tal fenômeno é problemático porque os gradientes muito pequenos podem levar a um treinamento ineficiente ou até mesmo à estagnação do processo de atualização dos pesos.

Algumas estratégias para mitigar ou evitar o problema do desaparecimento do gradiente incluem:

- uso de **funções de ativação** que mantêm os gradientes em uma faixa mais desejável, como ReLU (Rectified Linear Unit) e suas variantes;
- uso de **inicializações cuidadosas de pesos**, como a inicialização de He e a inicialização de Xavier;
- uso de **técnicas de normalização**, como a normalização de lotes e a normalização de lotes em camadas de ativação;
- uso de **arquiteturas de rede alternativas**, como redes residuais (ResNets) e redes neurais recorrentes com portas de memória, como as LSTMs (Long Short-Term Memory) e as GRUs (Gated Recurrent Units).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Essas estratégias ajudam a manter os gradientes em um intervalo mais saudável, evitando que diminuam rapidamente à medida que retrocedem nas camadas da rede.

2.4 Arquiteturas derivadas

Não podemos negar o sucesso e importância das RNNs. Então, para amenizar os problemas enfrentados, outras arquiteturas de redes recorrentes foram propostas, cada uma com suas características distintas e aplicações específicas. Vejamos as principais:

- **LSTM (Long Short-Term Memory)**: assim como a RNN padrão, a LSTM é projetada para processar sequências temporais e capturar dependências de longo prazo. No entanto, a LSTM possui uma arquitetura mais complexa, introduzindo unidades de memória que ajudam a superar o problema do desaparecimento do gradiente. Ela é composta por três portas principais – entrada, esquecimento e saída –, que controlam o fluxo de informação ao longo do tempo (Figura 6).
    - *Pontos fortes*: é altamente eficaz em capturar dependências de longo prazo em sequências temporais, mantendo a capacidade de aprender dependências de curto prazo.
    - *Pontos fracos*: pode ser mais computacionalmente intensiva do que a RNN padrão devido à sua arquitetura mais complexa, o que pode resultar em tempos de treinamento mais longos.
    - *Aplicações principais*: tradução automática, geração de legendas de imagens, análise de sentimento em texto.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

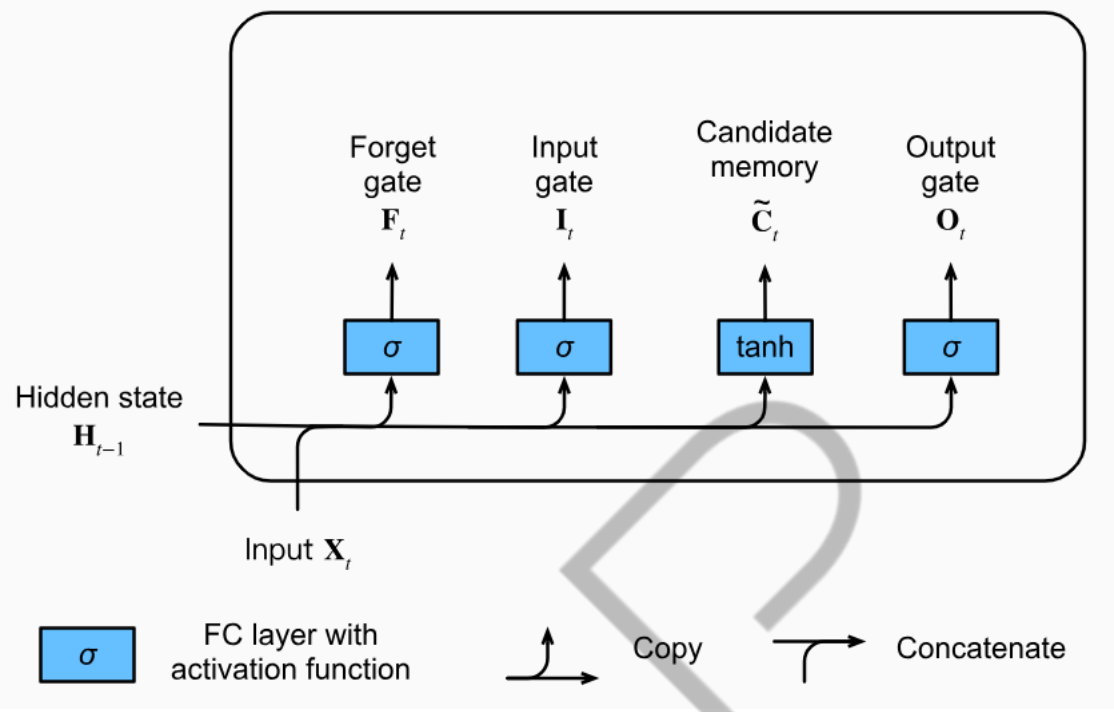


Figura 6 – Representação de um neurônio de uma LSTM
Fonte: Imagem reproduzida de: https://pt.d2l.ai/chapter_recurrent-modern/lstm.html

- **GRU (Gated Recurrent Unit)**: a GRU é uma variação da RNN que também opera em sequências temporais e é capaz de lidar com dependências de longo prazo. No entanto, possui uma estrutura mais simples do que a LSTM, com menos portas de controle de fluxo de informação. Ela combina as funções de atualização e esquecimento em uma única porta, tornando-a computacionalmente mais eficiente (Figura 7).
    - *Pontos fortes*: a GRU é eficaz em capturar dependências de longo prazo em sequências temporais, com menor complexidade computacional em comparação com a LSTM.
    - *Pontos fracos*: como possui menos portas de controle do que a LSTM, a GRU pode ser menos expressiva em algumas tarefas que exigem modelagem de dependências complexas de longo prazo.
    - *Aplicações principais*: reconhecimento de fala, tradução automática, análise de sentimentos em texto.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

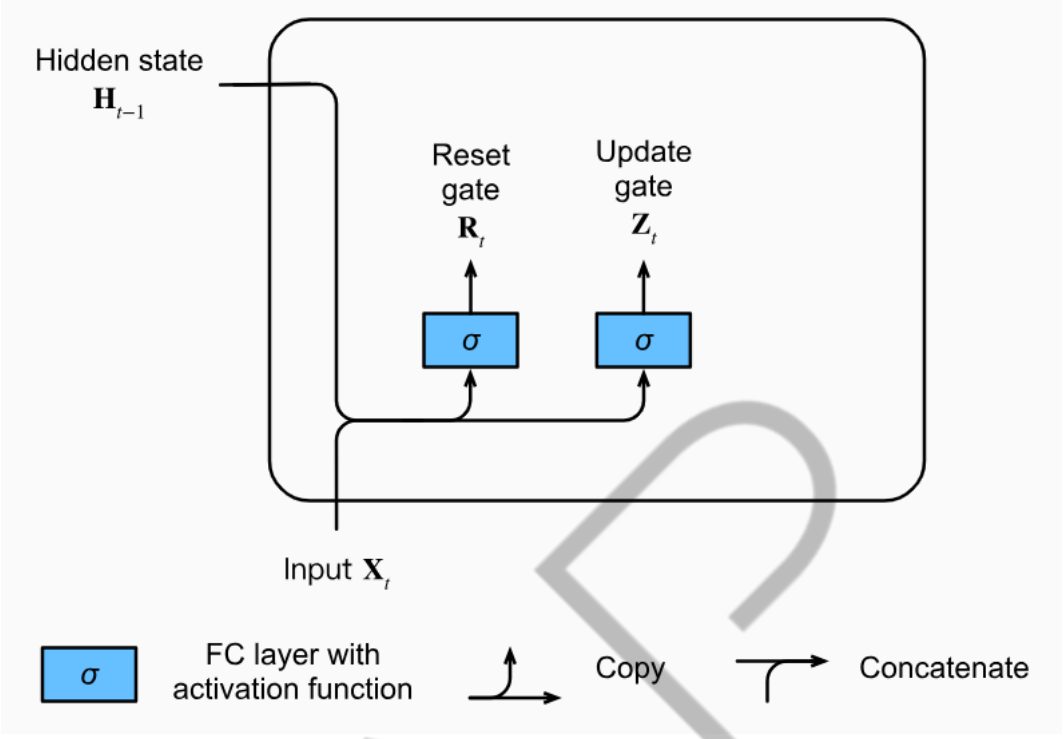


Figura 7 – Representação de um neurônio de uma GRU
Fonte: Imagem reproduzida de: https://pt.d2l.ai/chapter_recurrent-modern/gru.html

- **Bidirectional RNN**: assim como a RNN padrão, a Bidirectional RNN opera em sequências temporais, mas com a capacidade de considerar informações passadas e futuras simultaneamente. A Bidirectional RNN consiste em duas camadas de células RNN, uma processando a sequência temporal no sentido direto e a outra processando no sentido reverso. Isso permite que a rede capture informações contextuais de ambos os lados da sequência (Figura 8).
    - *Pontos fortes*: a Bidirectional RNN é eficaz em capturar contexto global em sequências temporais, levando em conta tanto o passado quanto o futuro.
    - *Pontos fracos*: pode ser mais computacionalmente intensiva do que a RNN padrão devido à necessidade de processar a sequência em ambos os sentidos.
    - *Aplicações principais*: reconhecimento de entidades nomeadas em linguagem natural, segmentação de fala em texto.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

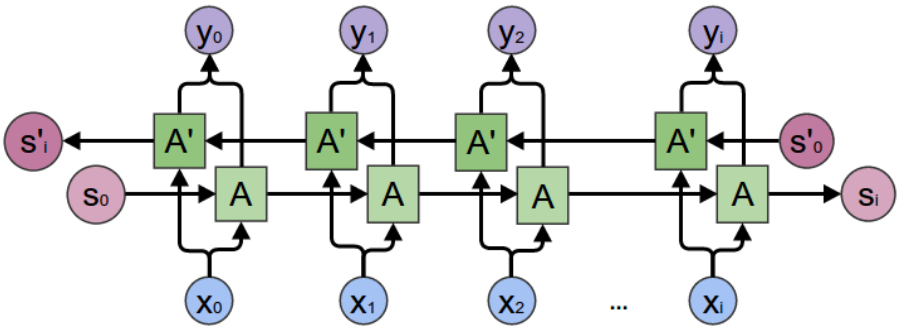


Figura 8 – Representação da estrutura geral de uma rede neural bidirecional
Fonte: Imagem reproduzida de: https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66

- **Encoder-Decoder (Seq2Seq)**: o Encoder-Decoder é uma estrutura composta por duas redes RNNs, uma atuando como codificador e outra como decodificador, que operam em sequências temporais. É projetado para lidar com tarefas de sequência para sequência, como tradução automática, onde uma sequência de entrada é codificada em um vetor de contexto e, em seguida, decodificada para uma sequência de saída em outra língua (outra representação) (Figura 9).
    - *Pontos fortes*: é altamente eficaz em tarefas de geração de sequência para sequência, como tradução automática e resumo de texto.
    - *Pontos fracos*: pode ser desafiador treinar efetivamente em conjuntos de dados grandes devido à necessidade de capturar informações contextuais de longo prazo.
    - *Aplicações principais*: tradução automática, geração de legendas de imagens, resumo automático de texto.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

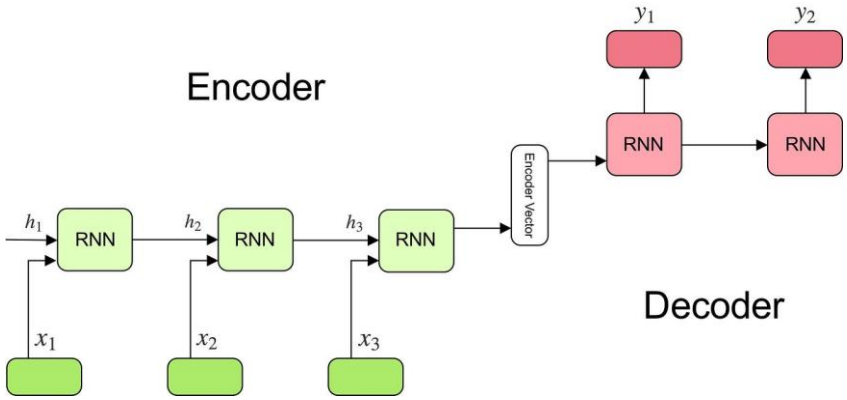


Figura 9 – Representação da estrutura geral de uma rede neural bidirecional
Fonte: Imagem reproduzida de: https://towardsdatascience.com/understanding-encoder-decoder-sequence-to-sequence-model-679e04af4346

Essas são apenas algumas das arquiteturas derivadas das RNNs, que ampliam suas capacidades e aplicações em várias áreas, oferecendo soluções para problemas específicos encontrados no processamento de sequências temporais. Cada uma dessas arquiteturas tem suas vantagens e limitações, e cabe lembrar que a escolha entre elas depende da natureza da tarefa e dos requisitos específicos do problema em questão.

3 ARQUITETURA LSTM

3.1 Introdução

Dentre as arquiteturas derivadas das RNNs, as LSTMs possuem destaque pela sua variedade de aplicações e por endereçar o problema de desaparecimento do gradiente.

Desenvolvida por Hochreiter e Schmidhuber em 1997, a LSTM foi projetada para capturar dependências temporais de longo prazo, preservando informações importantes ao longo de muitos passos no tempo. Ela introduz unidades de memória controladas por portas, que permitem que a rede aprenda quais informações reter e quais descartar durante o processo de treinamento. Isso a torna particularmente eficaz

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

em tarefas que exigem a modelagem de dependências temporais complexas, como tradução automática, geração de legendas de imagens e análise de séries temporais.

Nos últimos anos, houve avanços significativos na arquitetura e no treinamento de LSTMs, impulsionados principalmente pelo aumento do poder computacional e pelo desenvolvimento de técnicas de otimização mais eficazes. Pesquisadores têm explorado variações e melhorias na estrutura básica da LSTM, além de estratégias de treinamento mais avançadas, como o uso de algoritmos de otimização adaptativa e técnicas de regularização mais sofisticadas. Essas evoluções têm levado a melhorias significativas no desempenho e na eficiência das LSTMs, tornando-as uma escolha cada vez mais popular em uma ampla gama de aplicações de aprendizado de máquina.

3.2 Componentes das LSTMs

As LSTMs continuam sendo RNNs formadas por neurônios organizados em camadas conectadas. A grande diferença está em como os neurônios são formados. Nas LSTMs, cada célula neural é mais complexa, consistindo em quatro componentes principais: unidades de memória, portões de entrada, portões de esquecimento e portões de saída. A importância de cada componente é dada abaixo:

- **Unidades de Memória**: as unidades de memória em uma LSTM são responsáveis por armazenar e atualizar informações ao longo do tempo. Elas mantêm um estado de memória contínuo, que pode ser modificado por meio dos portões de entrada e de esquecimento;
- **Portões de Entrada (*Input Gates*)**: os portões de entrada controlam a quantidade de nova informação que será adicionada à memória atual. Eles decidem quais valores serão atualizados na memória com base na entrada atual e no estado anterior;
- **Portões de Esquecimento (*Forget Gates*)**: os portões de esquecimento determinam quais informações antigas na memória serão descartadas. Eles são responsáveis por decidir o que manter e o que esquecer, com base na relevância das informações para a tarefa atual;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Portões de Saída (*Output Gates*)**: os portões de saída regulam a quantidade de informação que será transmitida para a próxima camada da rede ou para a saída final. Eles decidem qual parte da memória atual será exposta como saída da célula LSTM.

Para programar e configurar uma célula LSTM, é necessário definir os pesos associados a cada um dos componentes mencionados acima. Esses pesos são ajustados durante o treinamento da rede por meio do algoritmo de retropropagação pelo tempo, que calcula os gradientes dos pesos em relação à função de perda ao longo de uma sequência de entrada. Durante o treinamento, o backpropagation temporal é responsável por propagar o erro ao longo do tempo e ajustar os pesos de cada componente da célula LSTM para minimizar a perda.

Os componentes de uma célula LSTM são altamente influenciados pelo BPTT, pois suas atualizações de peso dependem dos gradientes calculados durante o processo de retropropagação. Isso significa que o desempenho e o comportamento da célula LSTM são diretamente afetados pela eficácia do algoritmo de treinamento em aprender padrões nas sequências de entrada e saída. Uma configuração cuidadosa e um treinamento adequado são essenciais para obter o melhor desempenho de uma célula LSTM em uma determinada tarefa.

3.3 Fluxo de informações em uma célula LSTM

O processo final de uma célula LSTM envolve a combinação de entradas e saídas por meio de portões ponderados e funções de ativação. Os portões de entrada, esquecimento e saída calculam saídas ativadas sigmoidalmente a partir de combinações lineares das entradas e do estado oculto anterior. Essas saídas determinam quais informações serão adicionadas à memória da célula, quais serão mantidas ou esquecidas e quanto da memória atual será exposta como saída. As funções de ativação sigmoid permitem que esses portões controlem o fluxo de informações com base em valores entre 0 e 1, influenciando a magnitude das

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

contribuições das entradas e do estado oculto anterior na memória da célula e na saída final.

O resultado esperado do processo é uma saída representativa que capture dependências de longo prazo em sequências de dados. Isso é alcançado pela capacidade dos portões de controlar cuidadosamente quais informações são atualizadas, mantidas ou esquecidas na memória da célula, permitindo que a LSTM aprenda padrões complexos e retenha informações relevantes ao longo do tempo. A combinação ponderada das entradas e do estado oculto anterior, juntamente com as funções de ativação sigmoid, garante que a saída final da célula LSTM seja uma representação útil e informativa do contexto da sequência de entrada.

3.4 Melhoras práticas

Treinar uma LSTM, como toda rede neural artificial, requer uma série de cuidados para evitar problemas comuns como o overfitting. Para cada item selecionado, elencamos abaixo algumas estratégias a serem exploradas na definição da arquitetura da LSTM, bem como no seu treinamento:

- **Estratégias para prevenção de overfitting**:
  - *Aumento de Dados*:
    - Aplique técnicas de aumento de dados ajudando a expor o modelo a uma variedade maior de exemplos, reduzindo o risco de overfitting. Mas cuidado: as técnicas de aumentação devem preservar o aspecto temporal/sequencial dos dados.
  - *Early Stopping*:
    - Monitore o desempenho do modelo em um conjunto de validação e interrompa o treinamento quando o desempenho começar a piorar. Isso ajuda a encontrar o ponto ideal de generalização antes que o modelo comece a se ajustar demais aos dados de treinamento.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Estratégias para seleção de hiperparâmetros**:
  - *Grid Search e Random Search*:
    - Explore diferentes combinações de hiperparâmetros usando busca em grade ou busca aleatória. Isso permite encontrar a combinação ideal que otimiza o desempenho do modelo. A contrapartida é que o tempo de convergência dessas combinações pode ser muito alto.
  - *Cross-Validation*:
    - Utilize validação cruzada para avaliar o desempenho do modelo em diferentes conjuntos de dados. Isso ajuda a garantir que o modelo seja robusto e generalizável para dados não vistos.
- **Estratégias de regularização**:
  - *Dropout Recorrente*:
    - Aplique o dropout recorrente em unidades de LSTM para reduzir a coadaptação entre unidades de tempo consecutivas. Isso ajuda a evitar o overfitting e melhora a capacidade do modelo de generalizar para novos dados.
  - *Regularização de Peso*:
    - Use regularização de peso, como regularização L2, para penalizar pesos maiores e evitar a complexidade excessiva do modelo. Isso ajuda a evitar o overfitting e promove uma melhor generalização.
- **Estratégias para otimização de desempenho**:
  - *Utilização de GPUs*:
    - Aproveite o poder computacional das GPUs para acelerar o treinamento de modelos LSTM. Isso permite treinar modelos maiores em menos tempo e facilita a experimentação com diferentes arquiteturas.
  - *Paralelização de Treinamento*:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Explore técnicas de paralelização de treinamento para distribuir o processamento em várias GPUs ou dispositivos. Isso pode reduzir o tempo de treinamento e aumentar a eficiência do processo de treinamento.

Essas técnicas não garantem melhores soluções, mas seu emprego é altamente recomendado devido aos resultados empíricos, que sugerem potencial de ganho à performance das LSTMs. Ou seja, o sucesso de qualquer rede neural depende da sua arquitetura, dos dados e da complexidade do problema modelado. As dicas e melhores práticas listadas aqui indicam caminhos que podem favorecer os modelos propostos para as suas aplicações.

4 MÃO NA MASSA! IMPLEMENTANDO UMA LSTM

Agora que conhecemos as LSTMs, está na hora de praticarmos sua implementação. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab), usando o TensorFlow.

DICA: Veja o material HOW TO para acessar o ambiente do Colab.

DICA: Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 8) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

DICA: Para o código deste capítulo, utilize os arquivos de apoio "**LSTM_TATA_train.csv**" e "**LSTM_TATA_test.csv**".

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Sobre a base de dados:

A **Tata Steel Stock Dataset (2015-2021)**, disponível no Kaggle, é uma coleção de dados que contém informações históricas sobre os preços das ações da Tata Steel, uma empresa multinacional de fabricação de aço. As variáveis incluídas no conjunto de dados geralmente abrangem o preço de abertura, fechamento, alta, baixa, volume de negociação e possíveis indicadores técnicos. Esses dados são valiosos para a previsão de preços de ações usando modelos de Deep Learning, especialmente as RNNs, devido à natureza sequencial e dependente no tempo das séries temporais financeiras. Com base em padrões históricos, as RNNs podem capturar relações temporais complexas e fornecer previsões precisas sobre os preços futuros das ações da Tata Steel, auxiliando investidores e analistas na tomada de decisões informadas.

Vamos para o código!

Começaremos com os *imports*:

```
# Imports
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from keras.models import Sequential
from keras.layers import LSTM, Dropout, Dense

from tensorflow.keras import layers, models
```

Código-fonte 1 – Imports do código de LSTM
Fonte: Elaborado pelo autor (2024)

Esse primeiro código não gera saída. Vamos carregar o dataset:

```
# Leitura do dataset
df = pd.read_csv("LSTM_TATA_train.csv")
df
```

Código-fonte 2 – Carregando o dataset
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Date	Open	High	Low	Last	Close	Total Trade Quantity	Turnover (Lacs)
0	2018-09-28	234.05	235.95	230.20	233.50	233.75	3069914	7162.35
1	2018-09-27	234.55	236.80	231.10	233.80	233.25	5082859	11859.95
2	2018-09-26	240.00	240.00	232.50	235.00	234.25	2240909	5248.60
3	2018-09-25	233.30	236.75	232.00	236.25	236.10	2349368	5503.90
4	2018-09-24	233.55	239.20	230.75	234.00	233.30	3423509	7999.55
...	...	...	...	...	...	...	...	...
2030	2010-07-27	117.60	119.50	112.00	118.80	118.65	586100	694.98
2031	2010-07-26	120.10	121.00	117.10	117.10	117.60	658440	780.01
2032	2010-07-23	121.80	121.95	120.25	120.35	120.65	281312	340.31
2033	2010-07-22	120.30	122.00	120.25	120.75	120.90	293312	355.17
2034	2010-07-21	122.10	123.00	121.05	121.10	121.55	658666	803.56

2035 rows × 8 columns

Figura 10 – Saída do Código-fonte 2
Fonte: Elaborado pelo autor (2024)

A Figura 10 mostra que o arquivo de treino possui 2035 linhas com 8 colunas, cada uma representando uma variável. Dentre elas, a mais importante é a data (Date), indicando a temperabilidade daquela informação.

Nossa análise será predizer valores futuros da ação da empresa com base nos valores passados. Para isso, podemos utilizar qualquer coluna relativa ao valor da ação, desde que sigamos sempre com a mesma escolha. Por exemplo: podemos utilizar a coluna “Close” para analisar o histórico do valor da ação na abertura da bolsa e tentar prever os valores futuros da ação na abertura da bolsa. Não podemos misturar as colunas, pois a tendência perderia o sentido.

Sem perda de generalidade, vamos adotar como escolha do valor da ação seu preço na abertura da bolsa (Open). Nós sabemos que os dados já estão em ordem temporal, então, nosso “dataset de treino” será os valores dessa coluna. Assim, vamos selecionar o array numpy que com esses valores por meio do seguinte código:

```
# Puxando apenas a coluna do valor da ação na ABERTURA
training_set = df.iloc[:,1:2].values

type(training_set)
```

Código-fonte 3 – Selecionando apenas a coluna “Open”
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
numpy.ndarray
```

Figura 11 – Saída do Código-fonte 3
Fonte: Elaborado pelo autor (2024)

A Figura 11 mostra que nossa seleção consiste apenas de um vetor de informações, não mais uma tabela (pois utilizamos apenas 1 variável). Podemos analisar as tendências históricas das ações da Tata na abertura da bolsa por meio de um gráfico, gerado com o código a seguir:

```
# Plotando a série histórica
plt.figure(figsize=(20, 9))
plt.plot(range(df.shape[0]),
         df["Open"][-1::-1])
plt.title("Série temporal da ação")
plt.xticks(range(0, df["Date"].shape[0], 50),
           df["Date"][-1:0:-50],
           rotation=45)
plt.xlabel("Datas")
plt.ylabel("Preço de abertura")
plt.show()
```

Código-fonte 4 – Plotando a série histórica dos dados
Fonte: Elaborado pelo autor (2024)

Note que o código faz algumas “mágicas” para produzir um gráfico que faça sentido para nós, humanos. Os dados no arquivo estão do mais recente para o mais novo, mas o gráfico deve trazer as informações da mais antiga para a mais recente. Assim, a seleção dos dados ocorre em ordem inversa, da última para a primeira, pulando de 1 em 1 (o que justifica o *slicing* “`[-1::-1]`”).

A mesma lógica se aplica para o eixo X, das datas, mas para evitar um eixo muito poluído, plotamos os valores das datas apenas de 50 em 50 números, com uma rotação de 45º. O resultado está na Figura 12:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

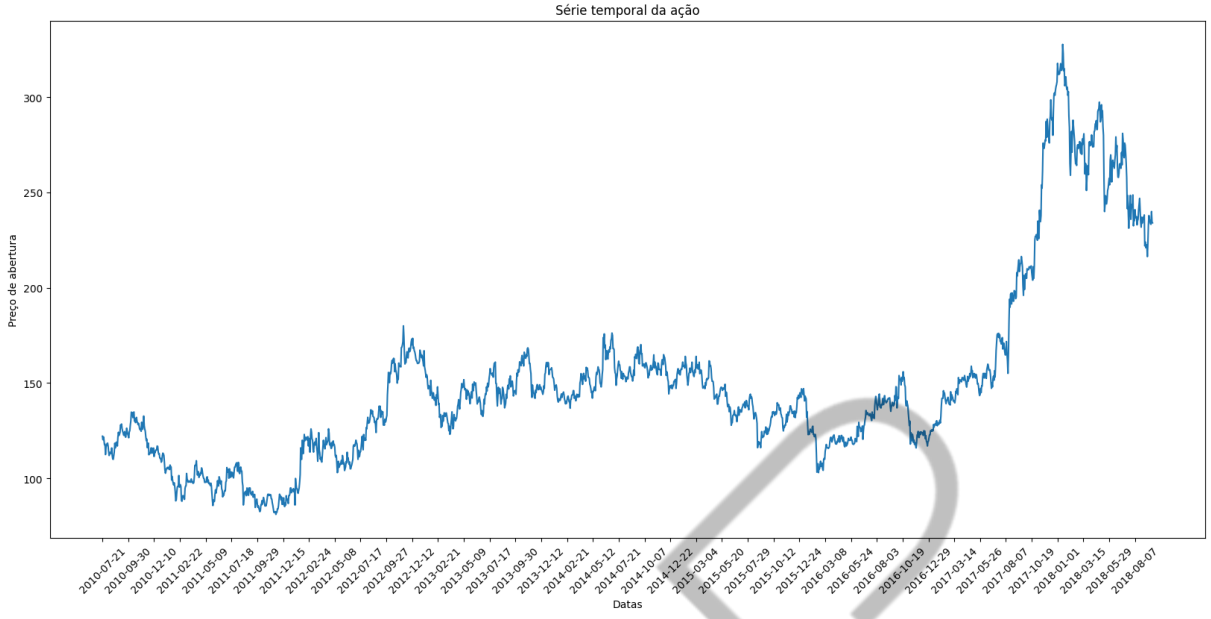


Figura 12 – Saída do Código-fonte 4
Fonte: Elaborado pelo autor (2024)

Podemos ver que em outubro/2017, houve um estouro do preço da ação, com consequentes quedas (apesar as oscilações) até agosto/2018.

Como estamos trabalhando com apenas 1 variável, ela claramente já está “escalada”, mas sabemos que as redes neurais performam melhor quando os dados estão expressos entre 0 e 1. Portanto, vamos padronizar os dados por meio do seguinte comando:

```
# Padronizando os dados
mm = MinMaxScaler()
training_set_scaled = mm.fit_transform(training_set)

training_set_scaled
```

Código-fonte 5 – Padronizando a escala dos dados entre 0 e 1
Fonte: Elaborado pelo autor (2024)

```
array([[0.6202352 ],
       [0.62226277],
       [0.64436334],
       ...,
       [0.16504461],
       [0.15896188],
       [0.16626115]])
```

Figura 13 – Saída do Código-fonte 5
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Agora estamos prontos para criar nosso dataset temporal! Vimos que as séries temporais não são expressas apenas por 1 valor, mas uma sequência de valores. Então, **cada exemplo do dataset de treinamento (e teste) precisa conter uma série de valores sequenciais. O label é o valor seguinte. Construímos o dataset por meio de uma janela deslizante sobre os dados.**

Imagine que a janela tenha tamanho 50. Os primeiros 50 números compõem 1 exemplo para a rede aprender, sendo que seu label será o 51º valor. O próximo exemplo vai do 1º valor até o 51º, sendo que seu label é o 52º valor. O próximo exemplo vai do 2º valor até o 52º, sendo que seu label é o 53º valor. E assim sucessivamente.

Vamos construir os exemplos e labels das ações utilizando uma janela deslizante de tamanho 60, percorrendo todos os dados disponíveis no arquivo "train". O código abaixo não gera saída.

```
# Criando o dataset da série temporal
X_train = []
y_train = []
for i in range(60, 2035):
  X_train.append( training_set_scaled[i-60:i, 0] )
  y_train.append( training_set_scaled[i, 0] )

X_train, y_train = np.array(X_train), np.array(y_train)
```

Código-fonte 6 – Lógica para criar o dataset da série temporal
Fonte: Elaborado pelo autor (2024)

Como nosso dado possui apenas 1 dimensão (apenas 1 variável), vamos reescalar a variável "`X_train`" com o código abaixo:

```
# Reescalando o formato do treino
X_train = np.reshape(X_train, (X_train.shape[0],
                               X_train.shape[1],
                               1))

X_train.shape
```

Código-fonte 7 – Reescalando o formato da variável de treino
Fonte: Elaborado pelo autor (2024)

O código-fonte 7 gera a saída da Figura 14, a qual aponta que agora a variável "`X_train`" possui 3 dimensões, sendo 1975 linhas (exemplos de treinamento), 60

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

colunas (cada exemplo possui 60 valores sequenciais) e 1 série de dados (estamos lidando com apenas 1 variável).

```
(1975, 60, 1)
```

Figura 14 – Saída do Código-fonte 7
Fonte: Elaborado pelo autor (2024)

Agora estamos prontos para definir, compilar e treinar a rede neural recorrente. Faremos isso por meio do seguinte código:

```
# Definindo a arquitetura da rede
reg = Sequential()

reg.add( LSTM(units=50,
              return_sequences=True,
              input_shape=(X_train.shape[1], 1)) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50,
              return_sequences=True) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50,
              return_sequences=True) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50) )
reg.add( Dropout(0.2) )

reg.add( Dense(units=1) )

# Compilando a rede montada
reg.compile(optimizer="adam", loss="mean_squared_error")

# Treinando a rede
with tf.device('/device:GPU:0'):
  reg.fit(X_train,
          y_train,
          epochs=100,
          batch_size=32)
```

Código-fonte 8 – Definindo, compilando e treinando a LSTM
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A arquitetura definida é uma LSTM composta por quatro camadas, cada uma com 50 unidades de memória, seguida por camadas de dropout para regularização. A estrutura segue o padrão "`return_sequences=True`", o que significa que cada camada LSTM retorna sequências para a próxima camada, mantendo a informação temporalmente. A camada final é uma camada densa com uma única unidade, que é usada para prever a saída desejada. O dropout é usado para mitigar o overfitting, desativando aleatoriamente unidades durante o treinamento para evitar a co-adaptação.

Na compilação da rede, o otimizador Adam é escolhido, que é uma escolha comum para problemas de regressão devido à sua eficácia geral. A função de perda escolhida é a MSE, que é apropriada para problemas de regressão, onde o objetivo é minimizar a diferença quadrada entre as previsões do modelo e os valores reais.

A rede é treinada com 100 épocas com um batch de tamanho 32. A GPU acelera o treinamento, aproveitando a capacidade de processamento paralelo para aumentar a eficiência computacional. Durante o treinamento, a rede é ajustada aos dados de treinamento (X_train e y_train) para aprender os padrões presentes nos dados e fazer previsões precisas.

O código-fonte 8 gera a saída parcialmente mostrada na Figura 15. Para acelerar o treinamento, não deixe de aplicar a GPU no treinamento da rede. Sua saída pode não ser idêntica, mas certamente será parecida.

```
Epoch 90/100
62/62 [==============================] - 1s 16ms/step - loss: 6.8978e-04
Epoch 91/100
62/62 [==============================] - 1s 20ms/step - loss: 6.5753e-04
Epoch 92/100
62/62 [==============================] - 1s 22ms/step - loss: 6.3354e-04
Epoch 93/100
62/62 [==============================] - 1s 22ms/step - loss: 6.2726e-04
Epoch 94/100
62/62 [==============================] - 1s 16ms/step - loss: 6.3746e-04
Epoch 95/100
62/62 [==============================] - 1s 16ms/step - loss: 6.2945e-04
Epoch 96/100
62/62 [==============================] - 1s 16ms/step - loss: 6.5931e-04
Epoch 97/100
62/62 [==============================] - 1s 16ms/step - loss: 7.2689e-04
Epoch 98/100
62/62 [==============================] - 1s 17ms/step - loss: 6.3233e-04
Epoch 99/100
62/62 [==============================] - 1s 16ms/step - loss: 7.2915e-04
Epoch 100/100
62/62 [==============================] - 1s 15ms/step - loss: 0.0010
```

Figura 15 – Saída do Código-fonte 8
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Uma vez com o modelo treinado, é hora de avaliar seu desempenho. Para isso, usaremos os dados do arquivo de teste. Ele contém outros meses de dados além de agosto/2018.

Além da leitura do arquivo “teste”, precisamos concatená-los aos dados de treinamento para garantir que a sequência temporal seja mantida. Os dados de teste, de fato, serão os últimos da concatenação, pulando o tamanho do arquivo de teste para trás e também pulando 60 posições anteriores (pois nossa janela deslizante é de tamanho 60).

Depois, precisamos preparar o conjunto de teste da mesma forma que preparamos o conjunto de treino.

```
# Lendo os dados de teste
df_test = pd.read_csv("LSTM_TATA_test.csv")
real_stock_price = df_test.iloc[:, 1:2].values

# Considerando que os dados são contínuos, temos que concatenar os
arquivos
dataset_total = pd.concat((df["Open"],
                           df_test["Open"]),
                           axis=0)

# Dos dados concatenados, os exemplos de teste estão no final da
# concatenação pulando os últimos 60, devido ao tamanho da nossa
# janela. Aplique a padronização também (MinMax)
inputs = dataset_total[len(dataset_total)-len(df_test)-60:].values
inputs = inputs.reshape(-1, 1)
inputs = mm.transform(inputs)

# Preparando os dados de teste nos mesmos moldes dos dados de
treinamento
X_test = []
for i in range(60, 76):
  X_test.append( inputs[i-60:i, 0] )

# Ajustando o tamanho do teste, nos mesmos moldes do treinamento
X_test = np.array(X_test)
X_test = np.reshape(X_test,
                    (X_test.shape[0], X_test.shape[1], 1))
```

Código-fonte 9 – Lendo e preparando os dados de teste
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O código acima não produz saídas, então, podemos ir para o próximo passo, que é gerar as predições (que estarão expressas entre 0 e 1) e daí revertê-las para a escala original com a transformação inversa do MinMaxScaler():

```
# Predizendo valores das ações
predicted_stock_prices = reg.predict(X_test)

# Voltando os valores preditos na escala [0, 1] para a escala original
predicted_stock_prices = mm.inverse_transform(predicted_stock_prices)
```

Código-fonte 10 – Predizendo ações a partir da série temporal
Fonte: Elaborado pelo autor (2024)

A saída é bem direta e mostra apenas a completude da predição.

```
1/1 [==============================] - 1s 1s/step
```

Figura 16 – Saída do Código-fonte 10
Fonte: Elaborado pelo autor (2024)

Para avaliarmos de fato o poder de predição do nosso modelo, vamos construir um gráfico que compara 2 séries temporais: uma com os dados reais das ações da Tata ao longo do tempo e outra com os valores preditos das ações pela nossa LSTM:

```
# Mostrando os resultados
plt.plot(real_stock_price,
         color="black",
         label="Ações da TATA")
plt.plot(predicted_stock_prices,
         color="green",
         label="Ações PREDITAS da TATA")
plt.title("Predição de ações da TATA Consultoria")
plt.xlabel("Tempo")
plt.xticks(range(0, df_test["Date"].shape[0]),
           df_test["Date"].loc[-1:0:-1],
           rotation=45)
plt.ylabel("Preço da ação")
plt.legend()
plt.show()
```

Código-fonte 11 – Predizendo ações a partir da série temporal
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 17 compara as duas séries temporais. Podemos notar que o poder preditivo do modelo começa “ruim” e vai melhorando com o tempo, seguindo uma curva que se assemelha à tendência real:

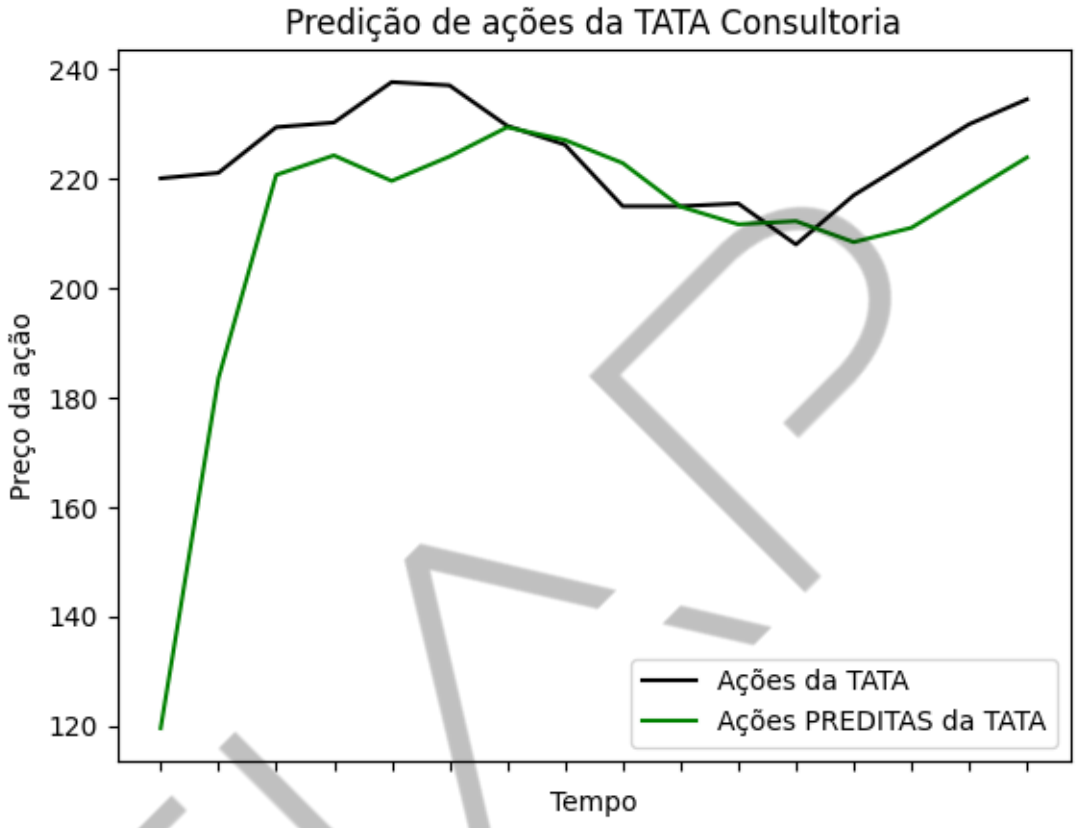


Figura 17 – Saída do Código-fonte 11
Fonte: Elaborado pelo autor (2024)

O último gráfico mostra que nosso modelo baseado em LSTM foi capaz de captar as tendências de variações do preço das ações da Tata. Ainda que não seja perfeito, ele é capaz de definir os momentos de queda e subida.

IMPORTANTE: Esse é um modelo simples que prevê o valor futuro de ações baseado apenas na série histórica das ações da mesma empresa. Na prática, a predição de valores financeiros é influenciada por uma série de fatores externos relativos ao mercado e à economia.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5 HANDS ON: PRATICANDO O USO DE LSTM

5.1 Seu desafio

Sua tarefa consiste em 2 etapas:

1. Complete o código da “mão na massa” para avaliar o erro do modelo, calculando o RMSE do modelo criado;
2. Crie um modelo de predição do valor da ação da Tata na abertura da bolsa, variando a arquitetura da LSTM, tamanho da janela deslizante e outros pontos que julgar necessário. Avalie seu modelo pelo RMSE também e compare com o modelo baseline criado anteriormente.

5.2 Proposta de solução

Abaixo, temos uma proposta de solução para o problema apresentado. Essa é apenas uma forma de resolver o problema. Seus modelos podem ser diferentes e com melhores/piores resultados: o importante é praticar a construção e treinamento de CNNs, bem como o uso de TensorFlow e GPU.

Parte 1: O modelo gerado possui um RMSE de 27,470005676288732.

```
# Calculando o RMSE
rmse = np.sqrt(np.mean((predicted_stock_prices -
real_stock_price)**2))
print("RMSE:", rmse)
```

Código-fonte 12 – Avaliando o RMSE do modelo anterior
Fonte: Elaborado pelo autor (2024)

Parte 2: Gerando um novo modelo. Esse possui um RMSE de 25,803656623317355.

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
from keras.models import Sequential
from keras.layers import LSTM, Dropout, Dense

# Leitura do dataset, seleção da variável de interesse e padronização
dos dados
df_train = pd.read_csv("LSTM_TATA_train.csv")
training_set = df_train.iloc[:, 1:2].values

mm = MinMaxScaler()
training_set_scaled = mm.fit_transform(training_set)


# Criando o dataset da série temporal
window_size = 100
X_train, y_train = [], []

for i in range(window_size, len(training_set_scaled)):
  X_train.append(training_set_scaled[i-window_size:i, 0])
  y_train.append(training_set_scaled[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)
X_train = np.reshape(X_train, (X_train.shape[0],
                               X_train.shape[1],
                               1))


# Definindo a arquitetura da rede
reg = Sequential()

reg.add(LSTM(units=100,
             return_sequences=True,
             input_shape=(X_train.shape[1], 1)))
reg.add(Dropout(0.2))

reg.add(LSTM(units=100,
             return_sequences=True))
reg.add(Dropout(0.2))

reg.add(LSTM(units=100,
             return_sequences=True))
reg.add(Dropout(0.2))

reg.add(LSTM(units=100))
reg.add(Dropout(0.2))

reg.add(Dense(units=1))

# Compilando a rede montada
reg.compile(optimizer="adam", loss="mean_squared_error")
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Treinando a rede
with tf.device('/device:GPU:0'):
  reg.fit(X_train,
          y_train,
          epochs=200,
          batch_size=32)


# Leitura e preparação dos dados de teste
df_test = pd.read_csv("LSTM_TATA_test.csv")
test_set = df_test.iloc[:, 1:2].values
test_set_scaled = mm.transform(test_set)
X_test, y_test = [], []

total_data = pd.concat((df_train["Open"], df_test["Open"]), axis=0)
inputs = total_data[len(total_data)-len(test_set_scaled)-
window_size:].values
inputs = inputs.reshape(-1, 1)
inputs = mm.transform(inputs)

for i in range(window_size, len(inputs)):
  X_test.append(inputs[i-window_size:i, 0])
  y_test.append(inputs[i, 0])

X_test, y_test = np.array(X_test), np.array(y_test)
X_test = np.reshape(X_test, (X_test.shape[0],
                             X_test.shape[1],
                             1))


# Predizendo valores das ações
predicted_stock_prices = reg.predict(X_test)

# Voltando os valores preditos na escala [0, 1] para a escala original
predicted_stock_prices = mm.inverse_transform(predicted_stock_prices)

# Mostrando os resultados
plt.plot(test_set,
         color="black",
         label="Ações da TATA")
plt.plot(predicted_stock_prices,
         color="green",
         label="Ações PREDITAS da TATA")
plt.title("Predição de ações da TATA Consultoria")
plt.xlabel("Tempo")
plt.xticks(range(0, df_test["Date"].shape[0]),
           df_test["Date"].loc[-1:0:-1],
           rotation=45)
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
plt.ylabel("Preço da ação")
plt.legend()
plt.show()

# Calculando o RMSE
rmse = np.sqrt(np.mean((predicted_stock_prices - test_set)**2))
print("RMSE:", rmse)
```

Código-fonte 13 – Proposta de solução do Hands On
Fonte: Elaborado pelo autor (2024)

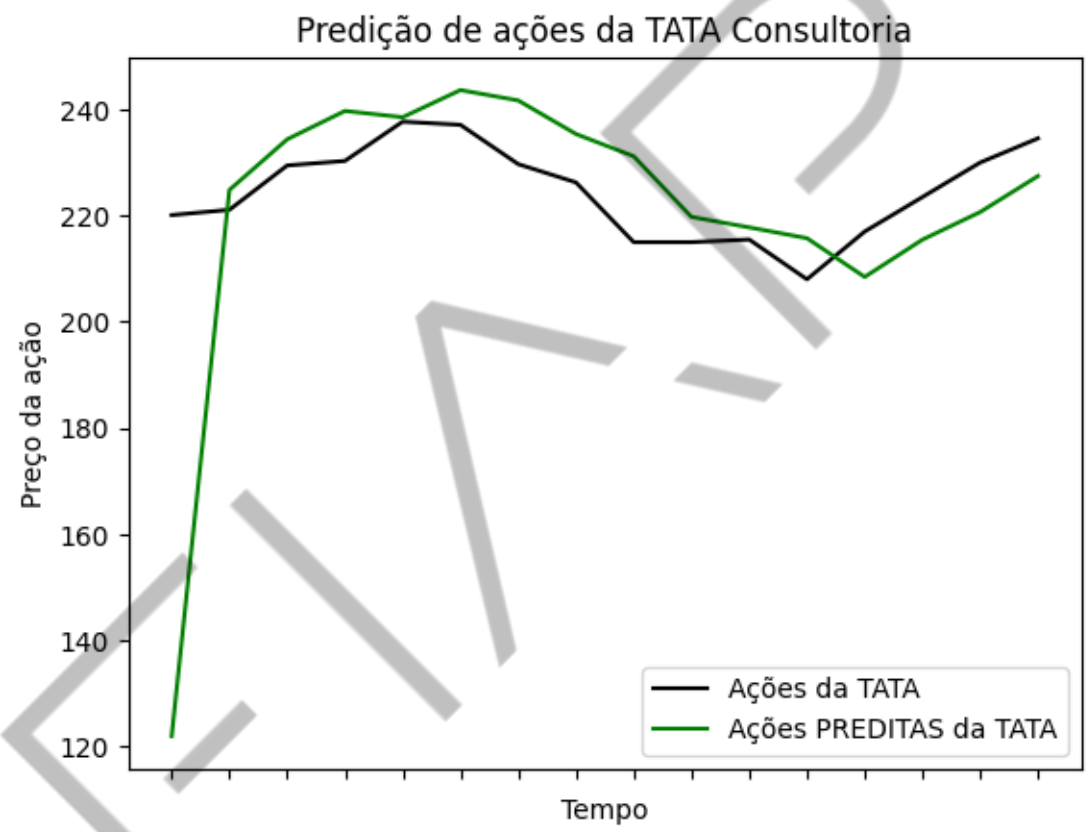


Figura 18 – Saída do Código-fonte 13
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BENGIO, Yoshua; GOODFELLOW, Ian; COURVILLE, Aaron. **Deep learning**. Cambridge, MA, USA: MIT press, 2017.

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow**. " O'Reilly Media, Inc.", 2022.

YU, Yong, et al. **A review of recurrent neural networks:** LSTM cells and network architectures. Neural computation, 2019, 31.7: 1235-1270.

SHERSTINSKY, Alex. **Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network**. Physica D: Nonlinear Phenomena, 2020, 404: 132306.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	**Explicação**
BPTT	*Backpropagation Through Time* – Retroalimentação dependente do tempo
CNN	*Convolutional Neural Network* – Redes Neurais Convolucionais
GRU	*Gated Recurrent Unit* – Unidade Recorrente Fechada
LSTM	*Long Short-Term Memory* – (Rede Neural de) Memória de Curto e Longo Prazo
RNN	*Recurrent Neural Network* – Redes Neurais Recorrentes

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com